An Empirical Analysis of Code Clones in GitHub Actions Workflows

Guillaume Cardoen
Software Engineering Lab
University of Mons
Mons, Belgium
guillaume.cardoen@umons.ac.be

Tom Mens
Software Engineering Lab
University of Mons
Mons, Belgium
tom.mens@umons.ac.be

Alexandre Decan
F.R.S.-FNRS Research Associate
Software Engineering Lab
University of Mons
Mons, Belgium
alexandre.decan@umons.ac.be

***Abstract*—Collaborative software development relies on social coding platforms that integrate CI/CD tools to automate repetitive development tasks. This article focuses on GitHub and its CI/CD service GitHub Actions, that supports and promotes reusable Actions and reusable workflows to avoid duplication of workflow configuration code. Despite these reuse mechanisms, copy-paste practices persist, leading to a high amount of Type I and Type II clones in workflow files. We conduct the first large-scale quantitative clone analysis across 118K GitHub Actions workflows in 34K GitHub repositories, containing 364K instances of clones. We study the prevalence of such clones in workflows, identify the specific workflow entities that are more prone to cloning, the distribution of clones at different levels of granularity, and the extent to which clones could be mitigated by the reuse mechanisms offered by GitHub. Our findings reveal that two-thirds of workflows contain clones, the majority of which could be refactored using built-in reuse mechanisms.**

***Index Terms*—GitHub Actions, CI/CD, clone detection, reusability, duplication**

I. Introduction

Collaborative software development is an essential practice for globally distributed project teams. It relies on social coding platforms such as GitHub, providing a multitude of collaboration tools such as version control, issue and pull request management, quality analysis, code reviewing and continuous integration/deployment (CI/CD). GitHub is the most popular social coding platform, with over 5.2 billion contributions by over 100 million users worldwide to more than 518 million repositories according to GitHub's 2024 Octoverse report.[1]

In November 2019, GitHub released GitHub Actions (abbreviated to *GHA* in the rest of this paper) as its built-in CI/CD tool, becoming the dominant CI/CD service in GitHub repositories in less than 18 months [1]. The GHA service requires repository maintainers to define and store *workflow configurations* (abbreviated to *workflows* hereafter) as YAML files in their repositories. Following the "configuration as code" paradigm, these workflows declare how to automate repetitive tasks (such as testing, building, issue triaging, quality analysis, deploying) in reaction to one or more events (*e.g.*, a push to a branch, a new pull request, a fixed schedule). When such an event occurs, a runner executes the corresponding workflow.

Workflows may use *Actions*, one of GHA's reusable components shared on the GitHub marketplace.[2] They can be written in JavaScript, refer to a Docker container or be composed themselves of other Actions and commands. Similarly, workflows may depend on entire *reusable workflows*. Despite these built-in reuse mechanisms, it has been shown that creating new workflows based on existing ones (including those from other developers) is common practice, and that copy-pasting from one's workflow is a frequent reuse mechanism [2]. This copy-paste reuse tends to lead to duplicated fragments within and across workflow files.

Such duplicated code is commonly referred to as *code clones* in traditional codebases. Code cloning has been, and remains to be, a very active topic of research [3], [4], [5], [6], [7]. Depending on the reasons of cloning, it may have positive effects or, to the contrary, be detrimental to the overall quality and maintainability of a codebase [8]. As GHA is widely used [1] as part of the supply chain of many software projects, there is a need to understand how cloning impacts GHA workflow maintainability, especially since the GHA documentation advocates to avoid duplication, notably by relying on reusable components such as Actions and reusable workflows.[3] A first step in this direction is to quantify the prevalence of code clones in GHA workflows.

This article therefore aims to shed more light on the prevalence and characteristics of code clones, referred to by *clones* in the rest of this paper, within and across workflows in GitHub repositories. More specifically, we report on a quantitative analysis of 364K+ instances of clones identified in a large dataset of 118K+ recently active workflows in 34K+ GitHub repositories to answer the following four research questions:

RQ1: *How prevalent are clones in workflows?* This question aims to establish to which extent clones occur in GHA workflows. We show that a majority of workflows contain clones.

RQ2: *Which parts of workflows are subject to clones?* This question aims to identify which parts of the workflows are more likely to be cloned. We show that clones appear in

[1] https://github.blog/news-insights/octoverse/octoverse-2024/

[2] https://github.com/marketplace?type=actions

[3] https://docs.github.com/en/actions/concepts/workflows-and-actions/reusing-workflow-configurations

all parts of a workflow, but primarily in jobs and steps, the main building blocks of workflows.

RQ3: *How are clones distributed across different scopes?* Copy-pasting may occur within the same workflow, across different workflows in the same GitHub repository or across different repositories, belonging or not to the same owner. This question aims to understand how clones are distributed across these different scopes. We show that clones are particularly present within a same repository.

RQ4: *To which extent could clones be avoided by using appropriate reuse mechanisms?* GHA offers mechanisms to avoid duplication through reusable Actions and workflows. This question assesses the potential of removing local clones in workflows by making use of these mechanisms. We show that these reuse mechanisms remain underutilised, despite their potential to remove a majority of clones.

The remainder of this paper is structured as follows. Section II provides the necessary background on code cloning and GHA to facilitate understanding what comes next. Section III discusses related research. Section IV explains the method that was followed to extract and process data to detect clones in GHA workflow files. Section V to Section VIII present the analysis and results for each research question. Section IX discusses the results, whereas Section X reports on the threats to validity of the performed research. Finally, Section XI concludes this paper and discusses avenues of future research.

II. Background

This section introduces the GHA workflow syntax and main concepts of code cloning.

A. About GHA workflows

A GitHub repository may use GHA to automate software workflows such as testing, building, and deployment. Workflows are declared in the .github/workflows directory, with each workflow defined in a YAML file that describes a sequence of automated tasks to be executed on GitHub's infrastructure. The human-readable YAML syntax used to encode such workflows is organized in a hierarchical tree structure composed of key-value pairs.

The left-hand side of Figure 1 shows an example of a workflow configuration. Each workflow must contain an on key (line 2) to declare the events that trigger the workflow execution, such as to declare that the workflow should run when commits are pushed or when a pull request is created.[4]

Each workflow also requires a jobs entry describing the list of jobs (at least one) that need to be executed when the workflow is triggered. For instance, lines 4-22 declare a single job (identified by the identifier lint on line 4). Lines 5-6 restrict the job's permissions to read-only access of the repository content. Lines 7-9 define a *matrix strategy* that allows running the job with different configurations. In this example, the os variable (line 9) allows to run the job on different operating systems through the runs-on key (line 10) that refers to this variable.

Each job is composed of a list of sequentially executed *steps*, in this example four steps. The first two, on lines 12-16, rely on a reusable JavaScript Action. A JavaScript Action is a single JavaScript file that is executed at runtime, possibly interacting with the GitHub API. Line 12 uses actions/checkout (version v4) to check out the git repository into the execution environment. Line 13-16 use actions/setup-python (version v5) to install and configure Python in this environment. It uses the with key to pass the python-version parameter that specifies which version needs to be installed. The third and fourth steps (lines 17-20 and 21-22, respectively) use the run key to specify shell commands. Lines 19-20 install Python's package manager pip and the project's development requirements, while line 22 runs the ruff Python linter.

Beyond the aforementioned JavaScript Actions, GHA offers two more reuse mechanisms (not used in the example). *Reusable workflows* share the syntax of regular workflows and allow reusing a same job within multiple workflows through the uses key of a job. *Composite Actions* can be used to encode a sequence of steps and are defined using a syntax similar to the one used in the steps section of a workflow. A workflow may then use such a composite Action as a single step, through the uses key of a step.

B. About code cloning

While no universal consensus exists on the precise definition of *clones*, they are commonly defined as code fragments that are similar according to some similarity measure [9], [10]. A *clone pair* represents two code fragments that are considered similar according to this measure. A *clone class* represents a set of fragments where each pair of fragments constitutes a clone pair.

Previous research on clones in general-purpose code usually classifies clones into four types [9], each corresponding to a different notion of similarity ranging from a nearly-exact textual match (Type I) to a semantic match (Type IV). *Type I* clones represent code fragments that are identical when ignoring comments or whitespaces. *Type II* clones extend Type I by also allowing differences in identifiers, literals, method names and types. *Type III* clones extend Type II by also allowing additions or deletions of code lines. *Type IV* clones, also known as *semantic clones*, represent code fragments achieving the same functional goal despite being syntactically different. Type IV clones are ignored in a lot of research because their detection is an undecidable problem.

Figure 1 provides four examples of code clones between both side, each identified by a different color. Type I clones are in blue and purple colors while Type II clones are in red and green colors. Section IV explains our definition of Type II clones in the case of workflows.

Depending on one's point of view, clones can be considered as either problematic or beneficial. On the negative side, clones tend to be considered as bad smells needing refactoring [11]. Clones may result in bug propagation, inconsistent bug fixes,

[4]The full list of workflow triggers can be found on https://docs.github.com/en/actions/reference/workflows-and-actions/events-that-trigger-workflows.

```yaml
name: Linting
on: [push, pull_request]
jobs:
  lint:
    permissions:
      content: read
    strategy:
      matrix:
        os: [windows-latest, ubuntu-latest, macos-latest]
    runs-on: ${{ matrix.os }}
    steps:
      - uses: actions/checkout@v4
      - name: Setup Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.10'
      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements-dev.txt
      - name: Python lint
        run: ruff check --output-format=github .
```

```yaml
name: Validate module imports
on: [push, pull_request]
jobs:
  tach:
    if: github.event.pull_request.draft == false
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Set up Python
        uses: actions/setup-python@v4
        with:
          python-version: "3.10"
      - name: Install dependencies
        run: pip install tach==0.13.1
      - name: Run tach
        run: |
          tach report pennylane/labs
          tach check
```

Fig. 1: Example of a GitHub Actions workflow on the left, containing clones with another workflow on the right. Two Type I clones are highlighted in blue and purple, and two Type II clones are highlighted in red and green.

reduced readability, increased code size, and may obscure the origin of the code [8], [12], [13]. On the positive side, clones can enhance code readability and robustness while reducing development time [8], [9], [13]. Moreover, in languages lacking robust reuse mechanisms, cloning may be the only viable option for extending or adding a functionality without reinventing the wheel.

III. Related Work

This section reviews prior work related to our study. Section III-A presents the related work on GHA, while Section III-B provides related work on code cloning.

A. Research on GHA

Several studies have examined the adoption and usage of GHA. Kinsman *et al.* [14] studied how developers use GHA in open-source software (OSS) projects, observing that GHA was positively received by GitHub users, and that its introduction was associated with a reduction in the number of commits and merged pull requests. Golzadeh *et al.* [1] quantified the usage and migration of multiple CI/CD services in 91,980 GitHub repositories, finding that GHA became the most prominent CI/CD service on GitHub only 18 months after its introduction, replacing Travis as the previous dominant player. Through qualitative interviews with 22 experienced developers, Rostami Mazrae *et al.* [15] observed the same migration trend from Travis towards GHA, as well as a common practice of simultaneously using multiple CI/CD services within the same project. Through a quantitative analysis of 29,778 active GitHub repositories, Decan *et al.* [16] observed that most of them use GHA workflows for development purposes and rely widely on reusable Actions, primarily on those provided by GitHub itself.

Other studies have focused on workflow maintenance and quality. Valenzuela-Toledo and Bergel [17] studied GHA workflow evolution in 10 repositories, finding that more than three out of four workflow modifications consisted of adding or modifying instructions. They concluded that the usage of linters and related tools is necessary to improve the creation and maintenance of workflows. Saroar and Nayebi [18] surveyed 90 GHA developers and users, observing that configuring and debugging workflows is difficult. They suggested that workflow generation tools might help address this issue.

Khatami *et al.* [19] investigated 10,012 workflow files from 83 repositories. They identified 22 potential smells, including 7 smells deemed relevant by developers, 9 that received mixed reactions, and 6 that were mostly rejected by developers. Decan *et al.* [20] studied to which extent workflow files may face potential security risks by depending on outdated Actions. They observed that most workflows use Actions that are outdated for several months. Valenzuela-Toledo *et al.* [21] empirically studied the effort needed to maintain GHA workflows. While workflow automation increases efficiency, maintaining these workflows requires additional manual work and resources over time.

Bouzenia and Pradel [22] quantitatively studied the resource usage and optimization opportunities of GHA workflows based on a dataset of 3.7M workflow job logs. They found that workflow usage imposes a significant cost and resource consumption, and that most resources are consumed for building and testing repositories. They suggested that workflow maintainers adopt known optimization strategies to reduce these costs.

Zheng *et al.* [23] manually analysed 375 workflow failures across 260 OSS Java projects, identifying several issues that developers encounter when maintaining workflows, including difficulties in testing and debugging workflow files.

B. Research on clones

Many studies have focused on clone detection [3], [4], [5]. This has resulted in a wide range of tools, such as

NiCad [24], CCFinder [25], SourcererCC [26], Deckard [27], srcClone [28], CCDive [29], MSCCD [30] and Tailor [31]. These tools can be categorized as text-based, token-based, tree-based, graph-based, metric-based, or learning-based [3]. For learning-based approaches, machine learning models are typically trained on large clone datasets such as BigCloneBench [32]. Hybrid tools combining these categories have also been proposed [33], [34].

Empirical studies have examined clones in various contexts. Pate *et al.* [35] presented a systematic literature review on clone evolution, while Mondal *et al.* [36] found that more than one in six code fragments where a bug fix occurred were affected by bug propagation through clones. Roy and Cordy [9] estimated that 5-20% of code in software systems is typically duplicated, with some cases reaching 50% depending on the programming language. Koschke [37] reported a similar range of 7-23%, with one extreme case of 59%.

Of particular relevance to our work, several studies have examined clones in configuration files. Estefó *et al.* [38] studied launch configuration files in the Robot Operating System, finding that almost half of these files contained at least one clone. McIntosh *et al.* [39] studied Type I clones in 3,872 OSS build system configuration files, finding that half of build logic lines are duplicated at least once in Java build systems. They also found that some build systems contained only a limited number of clones, suggesting that cloning may be avoidable in build system configuration files. Additionally, they observed that clone-related problems in general-purpose code, such as inconsistent changes and prolonged fixes [12], also affected build system files. These findings motivate our investigation of similar phenomena in GHA workflows.

While both GHA and code cloning have been studied extensively, we are not aware of any previous work that has specifically analysed clones in GHA workflow files, which is the focus of our study.

IV. Method

This section outlines the method used to detect and analyse clones in GHA workflows. The dataset is explained in Section IV-A, while Section IV-B describes the clone detection pipeline we implemented.

A. Dataset

In order to study clones in GHA workflows, we require a dataset with a recent snapshot of syntactically valid workflows in GitHub software repositories. The snapshot should be recent to focus on actively maintained workflows and to avoid bias due to the evolution of practices over time. The dataset should contain a large collection of workflows to ensure statistical representativeness of the results and to increase confidence in our analysis of the prevalence of clones.

We relied on the 2025-10-09 version of a dataset proposed by Cardoen *et al.* [40], containing the history of over 267K workflows from over 49K repositories.[5] We extracted the most recent snapshot of all workflows, corresponding to their state on 25 August 2025. We only considered workflows meeting the following criteria: (1) the workflow must be syntactically valid to ensure we analyse well-formed configuration files; (2) the last modification date of the workflow must be in 2025 to avoid unmaintained workflows. Applying these filters on the dataset resulted in a final dataset comprising 118,363 workflows from 34,244 repositories.

[5]https://doi.org/10.5281/zenodo.17301952

B. Clone detection

We follow a standard multiphase clone detection pipeline [9], adapted to GHA workflows. A first *preprocessing* phase filters out uninteresting parts of the code (such as comments) and transforms each workflow into an intermediate abstract syntax tree (AST) representation. A second *match detection* phase detects similar code fragments and groups them into clone classes. A final *postprocessing* phase eliminates uninteresting clones (such as very small and accidental clones) and subsumed clones.

Similarly to other research on clones in configuration files [41], we focus on Type I and Type II clones. Examples of such clones are highlighted in color in Figure 1. Type II clones permit differences that do not affect the functional behavior, such as renaming variables, functions, and literals in general-purpose code. However, in workflow files, not all literals can be modified without affecting behavior. For example, changing the literal `actions/setup-python@v5` to `actions/setup-java@v5` in Figure 1 would fundamentally change the Action's behavior, from installing Python to installing Java. In contrast, renaming a step (such as `Setup Python` in Figure 1) does not affect the workflow's behavior. Therefore, we define Type II clones in GHA workflows as allowing differences in job identifiers, step names, and Action versions. This definition remains coherent with the standard definition of Type II clones in general-purpose code while accounting for the specificities of GHA workflows.

We detail the multi-phase clone detection pipeline below:

Preprocessing: We parse each workflow towards its AST representation, providing a structured view of the workflow's hierarchical organisation. Each AST leaf corresponds to a *(path, value)* pair, where *path* is the sequence of keys and list indices needed to reach the actual *value* associated to the key in the YAML structure. Taking line 12 in Figure 1 as an example, the leaf representing `actions/checkout@v4` corresponds to the pair (`jobs.lint.steps.0.uses`, `actions/checkout@v4`), using dot notation to separate the components of a path. In order to capture Type I and Type II clones, the *path* components are normalised as follows: (i) user-defined job identifiers are replaced by static tokens, since these identifiers do not impact the workflow's functionality. Returning to our previous example, we replaced the job identifier `lint` by the static token `*`; (ii) each list index is replaced by a static token, allowing to detect similar code fragments even when they occur at different positions in a list. Taking back our previous example, we replaced the list index `0` by the static token `*`. And the *value* components are normalised as follows: (i) Action

version information is ignored, in order to detect duplication across different versions. For example, actions/checkout@v4 is simplified to actions/checkout; (ii) values representing user-defined names (such as step names or job names) are replaced with a static token, since such values do not affect the semantics of a workflow file.

Match detection: Similar to Merkle hash trees [42], we associate each AST leaf with a SHA256 hash computed from its normalised path and normalised value. Each internal AST node is then recursively associated with a hash computed from the list of hashes of its children. This allows us to efficiently find clones: since subtrees with identical hashes are equal, all clone instances can by identified by hashes appearing at least twice across all workflows. Each *clone class* corresponds to a group of subtrees sharing the same hash. This method does not directly identify clones consisting of sequences of consecutive steps. Instead, we detect clones consisting of individual steps, and we group them later in case there are consecutive cloned steps. The order of the keys in a GHA workflow has no consequence on its behaviour, with the notable exception of steps, whose the order defines the execution order. Therefore, we only consider the order of steps when finding clones, ignoring the order of other keys.

Postprocessing: A postprocessing phase is necessary to remove clones considered irrelevant, as they artificially inflate the number of clones and potentially introduce bias. We start by excluding *accidental clones*, by setting a minimum threshold on the *clone length*, measured as the number of AST leaves the clone contains. We manually inspected clones of various lengths to determine a suitable threshold. Clones with a length below 5 were predominantly considered by the authors to be trivial, in line with previous studies on clone where a threshold of 6 lines is used [43].

Next, we removed *subsumed clone classes* (*i.e.*, redundant clone classes fully contained within other clone classes, as in [25]). Consider the following three code fragments: abd, abe, and b, where each letter abstracts a unique value. This example results in three clone classes: C_1 for the ab fragment, C_2 for the a fragment, and C_3 for the b fragment.

$$\underbrace{\underline{a}\ \underline{b}\ d}_{C_1}\qquad \underbrace{\underline{a}\ \underline{b}\ e}_{C_1}\qquad \underline{b}$$

C_2 C_3 C_2 C_3 C_3

In this example, ab is a clone, but each of its parts (a and b) are also clones. The clone class for fragment ab implies two *subsumed* clone classes for its subparts: one for a and one for b. Following the approach of [25], we ignore clone classes composed only of subsumed clones. In the example, we ignore C_2 that contains only subsumed clones (every instance in C_2 is already included in C_1), but we retain C_3 because it contains at least one instance that is not subsumed (*i.e.*, the third b).

V. RQ1: How prevalent are clones in workflows?

A first step in understanding clones in workflows is to detect and characterise them. **RQ1** aims to quantify the prevalence of clones in GHA workflows. In total, we detected 364,475 clones distributed across 81,866 clone classes. These clones appear in 79,636 workflows, representing 67.3% of all workflows. This highlights the high prevalence of clones in GHA workflows.

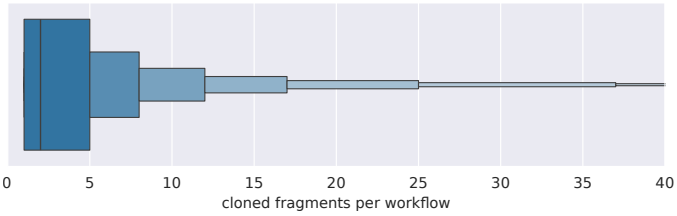

Fig. 2: Boxen plot of the distribution of the number of clone fragments per workflow.

For each workflow, we counted the number of clone fragments it contains. Figure 2 shows the distribution of the number of clone fragments per workflow. While clones are prevalent, most individual workflows contain a limited number of clone fragments. On average, a workflow contains 4.58 clone fragments, with a median of 2 clone fragments. Only 5.32% of workflows contain more than 13 clone fragments. Some outlier workflows (not shown in the figure) are more prone to cloning, containing up to 1,580 clone fragments. By manually inspecting some of these outliers, we observed that these workflows mostly contain variations of the same job with changes to the operating system or library versions, and many explicitly acknowledge in a YAML comment that they were automatically generated.

To understand the extent of duplication, we computed for each workflow the proportion of cloned values (relative to the total number of values in the workflow). For workflows containing at least one clone fragment, on average 50.7% of their values are cloned, with a median of 42.9%. This substantially exceeds the typical range of 5-25% observed in general-purpose code [9], [37], indicating that duplication is more widespread in GHA workflows.

Since clones may vary in length, we also analysed the length of clone fragments. We found that clones have an average length of 10.8 values and a median length of 7 values, and

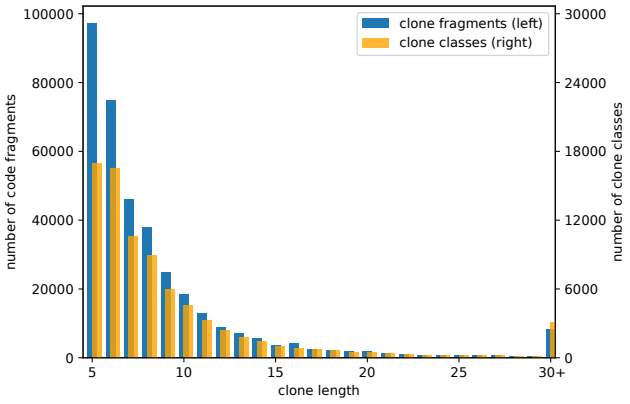

Fig. 3: Number of clone fragments (left axis) and clone classes (right axis) in function of clone length.

individually account on average for 18.7% (median 10.7%) of all values in the workflow they are part of.

Figure 3 shows the number of clone fragments and clone classes as a function of clone length. We observe that most clones and clone classes have a length between 5 (our minimum threshold) and 10 values. These cases account for 82.1% of all clones and 75% of all clone classes. In some cases, however, clones encompass a majority of the workflow values or even the entire workflow. For instance, we found 15,687 fully cloned workflows, representing 13.2% of all workflows. Of those fully cloned workflows, 85.5% are exact duplicates (including comments and whitespaces). The most frequent full workflow clone occurred 126 times across 123 repositories. We found this workflow exemplified multiple times on GitHub, including in the official documentation of GHA,[6] and in a previous version of a starter template provided by GitHub.[7]

Since a same code fragment may be cloned any number of times, we also analysed the clone class size, corresponding to how frequently each clone is duplicated. We found that a large majority of clone classes contain between 2 and 4 instances. The clone classes of size 2 represent 56.1% of all clone classes.

> Clones prevail in workflows. Clone classes tend to be small, with a majority containing only 2 clone fragments. Most clones span a limited number of workflows, occurring typically 2 to 4 times. Clones appear in two-thirds of all workflows, with a typical clone length between 5 and 10, representing 4% to 23% of the workflow length. In 13.2% of all cases, workflows are fully cloned.

VI. **RQ2**: WHICH PARTS OF WORKFLOWS ARE SUBJECT TO CLONES?

RQ2 aims to quantify which parts of workflows are cloned and how frequently they are. For each clone, we identify its root of the subtree representing the clone in the AST. Table I reports on these most common AST nodes, along with the proportion of clones they encompass. We only show nodes with a proportion of at least 0.5% for readability purposes. The majority of clones (81.1%) are found within job steps (*i.e.*, subsequences of one or more steps within a job). Within this category, sequences of two steps constitute the majority (55.0%) of clones. Sequences of three steps represent 23.0% of clones, and sequences of four steps represent 6.77%. Sequences of five steps or more represent only 15.2% of such clones.

Clones concerning a single step represent 12.9% of clones at the step level. The relatively low observed proportion of individual steps being cloned is a side-effect of the fact that they do not meet the minimal length threshold required to be considered as a clone. Many steps are very short, and less than one out of six steps in our dataset exceeds this threshold.

[6]https://docs.github.com/en/code-security/supply-chain-security/understanding-your-software-supply-chain/configuring-the-dependency-review-action

[7]https://github.com/actions/starter-workflows/commit/0239269003a81d1a264262c63fa8e90016003e10

TABLE I: Paths of the most frequent clones.

AST node		ratio
jobs.*.steps.*	sequence of steps of some job	81.1%
*	the entire workflow	4.30%
jobs.*.steps	all steps of some job	3.46%
jobs.*	some job of some workflow	3.38%
on	workflow trigger	1.34%
jobs	all jobs	1.22%
jobs.*.strategy	matrix strategy	1.03%
jobs.*.steps.*.env	environment variables of some step	0.65%

The second most widely cloned AST node corresponds to the entire workflow, accounting for 4.30% of all clones. Jobs are slightly less frequently cloned than steps, with 3.38% of clones involving a single job of a workflow and 1.22% concerning all jobs of a workflow. The remaining clones concern various other workflow constructs, such as workflow triggers (on, 1.34%), matrix strategies (jobs.*.strategy, 1.03%), environment variables defined at the step level (jobs.*.steps.*.env, 0.65%) and parameters defined at the step level (jobs.*.steps.*.with, 0.48%).

> 81.1% of clones in workflows are found within workflow steps, and contain subsequences of two to three steps. Many other workflow nodes are cloned as well, but to a lesser extent.

Not all AST clones appear in workflows. Consequently, and in order to get a better overview of which parts of workflows are more frequently cloned with respect to their actual usage, we computed the clone frequency of each AST node as being the number of times a node is part of a clone, divided by the number of times this node is actually used in a workflow. Figure 4 summarises the results in the form of a sunburst diagram. For readability, only nodes at depth six or less in the AST are reported, and all segments representing less than 5% of their parent segment are combined in an "other" category.

Starting from the center of the sunburst and moving outward, each layer represents a deeper level in the AST. One can observe that the root node (indicated by the # symbol) has a clone frequency of 13.3%, meaning that more than one out of eight workflows are cloned. At the next level, 15.4% of all jobs in a workflow are found cloned with at least another one, whereas a single job (indicated by <job_id >) has a clone frequency of 16.5%. These reported frequencies are quite close as a consequence of the fact that a majority of workflows (68.1%) only declare a single job. The set of all steps in a job has a clone frequency of 19.7%, whereas a single step (indicated by <step_id >) has a clone frequency of 42.5%. This indicates that steps are cloned more frequently than the higher-level jobs, and individual steps are more frequently cloned than entire sets of steps.

Going further within step, we observe that the two main mechanisms to specify how a step should be executed (*i.e.*, uses for reusable Actions or run for commands) exhibit different clone frequencies. For instance, run commands are

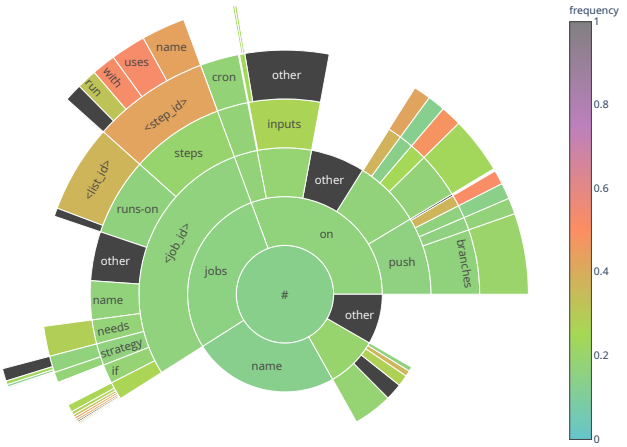

Fig. 4: Sunburst showing the clone frequency for every AST node. Each segment size reflects the proportion of duplicates within its parent node, while the colours of each segment represent the clone frequency of the AST node.

cloned in 32.1% of their occurrences, while the uses Actions are cloned in 52.2% of their occurrences. This difference is likely due to the length and variability of their contents.

The most cloned code fragment found in our dataset is a single step that uses the login-action Action to log in to the GitHub Container registry using the one's user credentials. This code fragment, shown in Figure 5 is cloned 1,430 times, perhaps because it is provided as an example on the official page of the Action.[8] The second most cloned code fragment, seen 816 times, represents the same step, except the username is specified by another variable github.repository_owner representing the name of the repository owner. Other commonly cloned code fragments include the initialization of both CodeQL[9] and autobuild (794 times) and the pull of the repository and initialization of CodeQL (740 times).

```
1 name: Login to GitHub Container Registry
2 uses: docker/login-action@v3
3 with:
4     registry: ghcr.io
5     username: ${{ github.actor }}
6     password: ${{ secrets.GITHUB_TOKEN }}
```

Fig. 5: Example of a cloned code fragment.

> 13.3% of all workflows are cloned, 16.5% of individual jobs are cloned, and 42.5% of individual steps are cloned. Reusable Actions are considerably more likely to be part of clones than user-defined run scripts.

VII. **RQ3**: How are clones distributed across different scopes?

The GHA language offers different reuse mechanisms, such as reusable Actions or reusable workflows, notably in order to avoid code duplication.[3] Actions that are stored in some public repository (and typically listed in the GitHub Marketplace) can be used in any workflow. In contrast, Actions that are stored in private repositories are mostly accessible to the owner of that repository, so only workflows belonging to the same repository or other repositories of the same owner can use these Actions.

Such restrictions imposed on the accessibility of certain reusable components limits the ability to avoid clones. To gain more insight in this phenomenon, **RQ3** studies the characteristics of clones in function of their *scope*. We distinguish four scope levels: (i) *file*: clones within the same workflow file (*e.g.*, a step that is duplicated in two different jobs of the same workflow); (ii) *repository*: clones between different workflows belonging to the same repository; (iii) *owner*: clones between workflows from different repositories belonging to the same owner; (iv) *all*: clones detected in **RQ1** without any constraint, for comparison purposes.

TABLE II: Comparison of clone characteristics across different scope levels.

	file	repository	owner	*all*
candidate workflows	118,363	105,674	61,209	118,363
workflows with clones	12,918	**40,238**	20,381	79,636
clones	100,143	**145,314**	65,358	364,475
clone occurrence rate	10.9%	**38.1%**	33.3%	67.1%
median clone coverage	24.7%	30.8%	**55.6%**	42.9%
median clone length	6	6	7	7

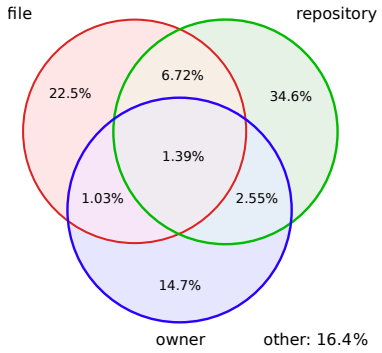

Fig. 6: A Venn diagram representing the proportion of clone classes belonging to different scopes.

The *all* scope contains all identified clone classes, with each class belonging to zero, one or more lower-level scopes. Figure 6 represents the proportion of clone classes belonging to those scopes. 45.3% of clone classes contain clones in the *repository* scope, 31.7% in the *file* scope, and 19.7% in the *owner* scope. Clone classes usually contain clones belonging to a same scope, although 8.11% (6.72%+1.39%) contain clones belonging to both the *file* and *repository* scopes.

> Clone classes tend to be well separated, with a low proportion of clone classes containing clones at different scope levels.

[8]https://github.com/docker/login-action
[9]https://codeql.github.com/

In order to characterise the distribution of clones per scope, we applied the clone detection pipeline of Section IV-B while considering only clones belonging to that scope. Table II summarises the findings.

The *repository* scope contains the highest number of workflows containing clones (40.2K), followed by the *owner* scope (20.3K) and the *file* scope (12.9K). In terms of number of clones, the order is different: the *repository* scope still comes first with 145.3K clones, but the *file* scope comes second with 100.1K clones and then the *owner* scope with 65.3K clones.

Note that these numbers of clones do not reflect the *clone occurrence rate* of each scope, which is the proportion of workflows with clones, w.r.t. to the number of *candidate* workflows. For example, the *repository* scope can only contain clones between different workflows for repositories that contains at least two workflows. This is not the case for all repositories, limiting the set of candidate workflows. Similarly, the *owner* scope requires workflows from different repositories belonging to the same owner, excluding owners with only one repository.

Table II reports the clone occurrence rate for each scope. The *repository* scope has the highest occurrence rate of 38.1%, followed by the *owner* scope with 33.3%. With 10.9% for the *file* scope, clones within the same workflow are 3 to 4 times less likely to appear than for the other scopes. Unsurprisingly, the *all* scope that does not impose any constraint on the candidate workflows has the highest clone occurrence rate of 67.1%.

In order to assess to which extent workflows are subject to clones, we computed the *clone coverage* for each workflow, defined as the proportion of AST leaves in the workflow that are part of a clone.[10] For each scope, we then computed the *median clone coverage* over all workflows with clones in that scope. The highest coverage is achieved at the *owner* scope, with a median of 55.6%. This is considerably higher than for the *repository* scope (30.8%) and the *file* scope (24.7%).

The fact that clones in the *owner* scope have a higher coverage but lower occurrence rate than in the *repository* scope could be due to a difference in *clone length* in both scopes. Indeed, the median clone length is 7 for the *owner* scope, while it is only 6 for the *repository* and *file* scopes, suggesting that the clone length is higher for wider scopes. We applied a Mann-Whitney U hypothesis test [44] for each pair of scopes to confirm that the *clone length* distributions were statistically different between the compared scopes. Using a significance level $\alpha = 0.001$ after Holm-Bonferroni correction for multiple comparisons, we could reject all null hypotheses, implying statistically significant differences between the distributions of the three scopes. We also assessed the effect size using Cliff's delta δ [45], [46], concluding a *small* difference between the *file* and *repository* scope ($\delta = 0.175$), but only a negligible difference ($\delta < 0.125$) for the other comparisons.

[10]The *clone coverage* metric is inspired by the *test coverage* metric in software development that computes the proportion of source code that is covered (*i.e.*, tested) by some unit test.

TABLE III: Usage of reusable components.

	reusable workflows	local Actions
workflows	9,672 (8.17%)	8,854 (7.48%)
repositories	3,859 (11.3%)	2,626 (7.67%)
occurrences	20,316 (3.21%)	28,640 (4.52%)
reused components	6,980	4,580

While clones are present in all scopes, the *repository* scope contains the most clones and the highest clone occurrence rate. The *owner* has the highest median clone coverage. Differences in clone length are small or negligible across scopes.

VIII. **RQ4**: To which extent could clones be avoided by using appropriate reuse mechanisms?

GHA provides and encourages the use of multiple reuse mechanisms to avoid code duplication, such as *reusable workflows* and *reusable Actions*. These reusable components can be stored either in the same repository as where the workflow is located, or in a different repository that may belong to the same or a different owner. *Reusable workflows* use a similar syntax as regular workflows and allow to reuse one or more jobs across multiple workflows. *Reusable Actions* allow to reuse a specific task declared in an external component, either as JavaScript code, a Docker container, or a so-called *composite Action* that uses a similar syntax as regular workflows to group multiple steps together. **RQ4** aims to simulate the potential of removing clones in workflows by relying on these built-in reuse mechanisms. The motivation for doing so is that it will decrease maintenance effort of workflows.

Considering the findings of **RQ2** that the majority of the clones in workflows are found within workflow steps, and contain subsequences of at least two steps, we focus on two frequent cases of clones: **(a)** clones corresponding to all steps of a workflow job could be moved into a *reusable workflow*; **(b)** clones corresponding to a sequence of at least two (but not necessarily all) steps within a workflow job could be moved into a reusable *composite Action*. Strictly speaking, we could also use composite Actions to refactor the clones of case **(a)**, but we prefer to keep both cases disjoint to facilitate the interpretation of the analysis.

We restrict the analysis to *local* clones found within a same workflow or repository and their potential replacement by reusable components (*i.e.*, reusable workflows or composite Actions) that will be stored in the same repository as the one in which the clones were found. This focus on repository-level clone removal is motivated by the fact that the repository owner remains in full control of the reusable component, avoiding the need for additional permissions from external repositories and avoiding additional dependencies.

In order to assess to which extent workflows are already relying on the usage of local reusable components, we quantify their presence in our workflow dataset. Table III reports on the usage of reusable workflows and other local reusable Actions. We found 20K occurrences of reusable workflow usage by 9.6K workflows stored in 3.8K repositories. These reusable

TABLE IV: Potential of removing local code clones by introducing reusable workflows or composite Actions.

	scope	potential of using reusable workflows	potential of using composite Actions	before	after
clones	file	2.22%	83.7%	100,143	14,124
	repository	2.26%	75.6%	145,314	32,211
workflows	file	2.00%	69.8%	12,918	3,402
	repository	1.25%	55.5%	40,238	16,816

workflows represent only 3.21% of all occurrences of the uses key in workflows.[11] We also found 28.6K occurrences of using *local* Actions by 8.8K workflows in 2.6K repositories, representing 4.52% of all occurrences of the uses key in workflows. Despite this higher number of occurrences of local Actions than reusable workflow uses, less workflows and repositories make use of local Actions. This analysis suggests that local reusable workflows and Actions are already used in practice to avoid code duplication, but only by a small fraction of workflows and repositories, thus leaving a lot of untapped potential for further clone removal.

> Despite their promise of reducing code duplication, the GHA reuse mechanisms remain underutilised by workflows.

In order to estimate to which extent the GHA reuse mechanisms (of reusable workflows and composite Actions) could be used to their full potential to further reduce code duplication within and across workflows, we quantified the opportunity for local refactoring for the cases **(a)** and **(b)** of cloned step sequences presented earlier. This simulates a best-case scenario where every such clone case would be refactored into a local reusable component.[12]

Table IV summarizes the results of the analysis. The potential of using *reusable workflows* to refactor entire cloned step sequences (within a workflow job) remains limited: only a small fraction of all clones (2.22% to 2.26%) would be targeted by this opportunity. On the other hand, *composite Actions* targeting clones consisting of at least two consecutive steps could potentially remove a majority of clones in the *file* scope (83.7%) and *repository* scope (75.6%). Combining both reuse mechanisms could potentially remove up to 85.9% of clones in the workflows, reducing their number from 100,143 to only 14,124 in the *file* scope, and from 145,314 to 32,211 in the *repository* scope. As a consequence, in the *file* scope, the number of workflows containing clones could decrease by 2.0% when using reusable workflows, and by 69.8% when using composite Actions. In the *repository* scope, the number of workflows containing clones could decrease by 1.25% when using reusable workflows, and by 55.5% when using composite Actions. Combining both reuse mechanisms could result in a reduction of clones in 9,516 workflows (73.7% of workflows with clones) in the *file* scope, and 23,422 workflows (41.8% of workflows with clones) in the *repository* scope.

The considerably higher potential to reduce clones through composite Actions than through reusable workflows aligns with the findings of **RQ2**, where the majority of clones concern strict subsequences of steps. This prevents them to be refactored into a reusable workflow (that requires to reuse every step), while this limitation is absent for composite Actions.

> While there is limited potential of using reusable workflows for removing code clones, the use of local composite Actions could potentially reduce up to 85.9% of all clones in a same workflow and up to 77.8% of all clones within the same repository.

IX. Discussion

The results of **RQ4** revealed quite some potential to replace code clones by either reusable workflows or by local composite Actions. This potential could be tapped by further promotion and automation of systematic reuse over copy-paste practices. First, GitHub could extend its Marketplace to become a generic reusable component library, not only encompassing reusable Actions, but also including reusable workflows and even starter workflow templates. None of them are currently supported or offered by the Marketplace, and the search criteria of the Marketplace remain quite limited today.

Second, automated duplication detection and refactoring tools could be proposed to detect clones in workflows, suggest opportunities for replacing them by the most appropriate reuse mechanism, and actually apply these refactorings in a context-aware manner. Such tools could either work locally to avoid external dependencies, or integrate with the Marketplace to select and suggest relevant refactoring opportunities, and to submit new reusable components to the Marketplace.

While this article focused on the well-known reuse mechanisms of Actions and reusable workflows, workflow maintainers can resort to other reuse mechanisms for creating their workflows. Studying the impact of those mechanisms is left for future work. One such mechanism is the use of *workflow templates* (a.k.a. *starter workflows*). The GitHub UI allows repository maintainers to create new workflows based on such templates, making them a likely source of nearly exact workflow clones. As explained in **RQ1**, the most duplicated workflow (seen 126 times) is part of the official GHA documentation and represents a previous starter workflow. Another frequent source of duplicated workflows are webpages or repositories hosting *example workflows*. When analysing the workflow clones in our dataset, we observed that the third and fifth most duplicated workflows (seen 51 and 46 times, respectively) are nearly exact copies of such workflow examples.[13]

[11]The uses key is required by the GitHub Actions syntax to refer to reusable components such as reusable workflows or reusable Actions.

[12]In practice, not all such clone occurrences may be undesirable, and there may be valid cases for preferring to keep those clones.

[13]These examples were found on https://github.com/r-lib/actions/tree/v2/examples.

Promoting and distributing example and template workflows in a central Marketplace further increase the popularity of such alternative reuse mechanisms. Combined with automated traceability support, it could help mitigating the duplicated effort that comes with the need to update one's workflow if the template it originates from contains bugs or security vulnerabilities. Such traceability would enable tools for change propagation and co-evolution of related clones. To come to such tooling, more empirical studies on the evolution of workflow clones are needed, in line with existing studies of clone genealogy [47] (*i.e.*, the history of changes of clones). Such studies identify the origin of clones, leading to a better understanding of their impact. They could also lead to recommendations or tools that mitigate adverse effects, such as the technical lag that workflows may accumulate over time due to clones present in them.

X. Threats to Validity

We follow the structure recommended by Wohlin *et al.* [48] to discuss possible threats to validity of our research.

Construct validity concerns the relation between the theory behind the experiment and the observed findings. We considered workflows as any file being grammatically valid. However, we did not consider if the workflows were used or executable in practice. We also removed subsumed clone classes to avoid biasing the analysis towards small clones. While we may still consider some subsumed clone pairs impacting the detected clone classes, it is unlikely to affect our conclusions that are mostly based on clone characteristics rather than on the clone classes. Finally, we only looked for Type I and Type II clones in workflows, leaving a study of Type III and Type IV clones as future work.

Internal validity concerns factors that could influence the observed study outcomes, independent of the intended manipulations or measurements. We used a minimal clone length of 5 to exclude irrelevant clones, based on manual judgment that clones of shorter lengths were mainly trivial. Starting from length 5, interesting clones started to appear, but we cannot avoid that some accidental clones above this threshold may still be part of our dataset. We also relied on a SHA256 hash function to detect clones. It is unlikely that this would have affected the results, since the probability of having hash collisions is astronomically low.

Conclusion validity addresses the degree to which the conclusions drawn from our analysis are reasonable. **RQ4** made the optimistic assumption that clones in workflow steps are a proxy of copy-paste reuse, allowing to remove such clones by other GHA reuse mechanisms. In practice, however, some of these clones may be intentional (*c.f.* [8]) or may have been generated through automated tools, making it less useful to replace such clones by proper reuse mechanisms.

External validity concerns the extent to which the study findings can be generalised or extended beyond the specific research boundaries. We relied on a dataset of public GitHub repositories with at least 100 stars, 300 commits, and that had at least one commit since August 25th, 2024. These criteria were used to exclude repositories that are not representative of typical software development practices, such as abandoned, personal, or experimental repositories [49]. While we share the rationale behind these criteria, it implies that the findings may not generalise to smaller or less active repositories that may employ workflows for different purposes, such as publishing GitHub Pages or managing personal projects. Similarly, we cannot claim that our findings generalise to workflows hosted in private repositories.

XI. Conclusion

GitHub Actions is the *de facto* workflow automation system for GitHub repositories. While it promotes several reuse mechanisms to avoid code duplication in workflow configurations, we provided empirical evidence that the potential of such reuse remain underutilised. We quantitatively studied 352K+ instances of clones in 118K+ workflows from 34K+ GitHub repositories, reporting on the current situation of cloning. The analysis revealed that clones remain omnipresent, with two-thirds of all workflows containing clones, more than 13% of all workflows being completely cloned, and workflows containing clones representing between 4% and 23% of the workflow most of the time. Clones are particularly present in workflow steps, with more than 4 out of 5 clones concerning subsequences of steps. Clones are also prominent within a same repository or workflow, with 45% and 32% of clone classes having clones in either scope, respectively. Through an optimistic "what-if" simulation, we highlighted that up to four out of five clones could potentially be removed by replacing them by one of GitHub's reuse mechanisms.

These results call for more studies on the evolution and genealogy of workflow clones, as well as increased awareness and better tool support for GitHub's workflow reuse mechanisms, as well as for clone refactoring and traceability support.

Acknowledgment and Data Availability

This work is supported by research project ARC-21/25 UMONS3 and by F.R.S.-FNRS under grant numbers J.0147.24 and F.4515.23. The code, data and scripts used in this paper are available on Figshare: doi.org/10.6084/m9.figshare.30363610

References

[1] M. Golzadeh, A. Decan, and T. Mens, "On the rise and fall of CI services in GitHub," in *International Conference on Software Analysis, Evolution and Reengineering*. IEEE, 2022.

[2] H. Onsori Delicheh, G. Cardoen, A. Decan, and T. Mens, "Automation and Reuse Practices in GitHub Actions Workflows: A Practitioner's Perspective," 2026, arXiv Preprint. [Online]. Available: https://doi.org/10.48550/arXiv.2601.11299

[3] Q. U. Ain, W. H. Butt, M. W. Anwar, F. Azam, and B. Maqbool, "A systematic review on code clone detection," *IEEE Access*, vol. 7, 2019.

[4] G. Lacerda, F. Petrillo, M. Pimenta, and Y. G. Guéhéneuc, "Code smells and refactoring: A tertiary systematic review of challenges and observations," *Journal of Systems and Software*, vol. 167, 2020.

[5] M. Lei, H. Li, J. Li, N. Aundhkar, and D.-K. Kim, "Deep learning application on code clone detection: A review of current knowledge," *Journal of Systems and Software*, vol. 184, 2022.

[6] M. Zakeri-Nasrabadi, S. Parsa, M. Ramezani, C. Roy, and M. Ekhtiarzadeh, "A systematic literature review on source code similarity measurement and clone detection: Techniques, applications, and challenges," *Journal of Systems and Software*, 2023.

[7] Z. Zhang and T. Saber, "Machine learning approaches to code similarity measurement: A systematic review," *IEEE Access*, 2025.

[8] C. J. Kapser and M. W. Godfrey, ""Cloning considered harmful" considered harmful: patterns of cloning in software," *Empirical Software Engineering*, vol. 13, pp. 645–692, 2008.

[9] C. K. Roy and J. R. Cordy, "A survey on software clone detection research," Queen's School of computing, Tech. Rep. 115, 2007.

[10] I. D. Baxter, A. Yahin, L. Moura, M. Sant'Anna, and L. Bier, "Clone detection using abstract syntax trees," in *International Conference on Software Maintenance*. IEEE, 1998, pp. 368–377.

[11] M. Fowler, *Refactoring: improving the design of existing code*. Addison-Wesley Professional, 2018.

[12] E. Juergens, F. Deissenboeck, B. Hummel, and S. Wagner, "Do code clones matter?" in *International Conference on Software Engineering*. IEEE, 2009.

[13] N. Saini, S. Singh *et al.*, "Code clones: Detection and management," *Procedia computer science*, vol. 132, pp. 718–727, 2018.

[14] T. Kinsman, M. Wessel, M. A. Gerosa, and C. Treude, "How do software developers use GitHub Actions to automate their workflows?" in *International Conference on Mining Software Repositories*. IEEE, 2021, pp. 420–431.

[15] P. Rostami Mazrae, T. Mens, M. Golzadeh, and A. Decan, "On the usage, co-usage and migration of CI/CD tools: A qualitative analysis," *Empirical Software Engineering*, vol. 28, no. 2, 2023.

[16] A. Decan, T. Mens, P. R. Mazrae, and M. Golzadeh, "On the use of GitHub Actions in software development repositories," in *International Conference on Software Maintenance and Evolution*. IEEE, 2022.

[17] P. Valenzuela-Toledo and A. Bergel, "Evolution of GitHub Action workflows," in *International Conference on Software Analysis, Evolution and Reengineering*. IEEE, 2022.

[18] S. G. Saroar and M. Nayebi, "Developers' perception of Github Actions: A survey analysis," in *International Conference on Evaluation and Assessment in Software Engineering*. ACM, 2023.

[19] A. Khatami, C. Willekens, and A. Zaidman, "Catching smells in the act: A GitHub Actions workflow investigation," in *International Working Conference on Source Code Analysis and Manipulation*. IEEE, 2024.

[20] A. Decan, T. Mens, and H. Onsori Delicheh, "On the outdatedness of workflows in the GitHub Actions ecosystem," *Journal of Systems and Software*, vol. 206, 2023.

[21] P. Valenzuela-Toledo, A. Bergel, T. Kehrer, and O. Nierstrasz, "The hidden costs of automation: An empirical study on Github Actions workflow maintenance," in *International Conference on Source Code Analysis and Manipulation*. IEEE, 2024.

[22] I. Bouzenia and M. Pradel, "Resource usage and optimization opportunities in workflows of Github Actions," in *International Conference on Software Engineering*, 2024.

[23] L. Zheng, S. Li, X. Huang, J. Huang, B. Lin, J. Chen, and J. Xuan, "Why do GitHub Actions workflows fail? an empirical study," *ACM Trans. Softw. Eng. Methodol.*, 2025.

[24] C. K. Roy and J. R. Cordy, "NICAD: Accurate detection of near-miss intentional clones using flexible pretty-printing and code normalization," in *International Conference on Program Comprehension*. IEEE, 2008.

[25] T. Kamiya, S. Kusumoto, and K. Inoue, "CCFinder: A multilinguistic token-based code clone detection system for large scale source code," *IEEE Transactions on Software Engineering*, vol. 28, no. 7, 2002.

[26] H. Sajnani, V. Saini, J. Svajlenko, C. K. Roy, and C. V. Lopes, "SourcererCC: Scaling code clone detection to big-code," in *International Conference on Software Engineering*, 2016.

[27] L. Jiang, G. Misherghi, Z. Su, and S. Glondu, "Deckard: Scalable and accurate tree-based detection of code clones," in *International Conference on Software Engineering*. IEEE, 2007.

[28] H. W. Alomari and M. Stephan, "Clone detection through srcClone: A program slicing based approach," *Journal of Systems and Software*, vol. 184, February 2022.

[29] Y. Glani, L. Ping, S. A. Shah, and L. Ke, "CCDive: A deep dive into code clone detection using local sequence alignment," *Tsinghua Science and Technology*, vol. 30, no. 4, pp. 1435–1456, 2025.

[30] W. Zhu, N. Yoshida, T. Kamiya, E. Choi, and H. Takada, "Development and benchmarking of multilingual code clone detector," *Journal of Systems and Software*, vol. 219, 2025.

[31] J. Liu, J. Zeng, X. Wang, and Z. Liang, "Learning graph-based code representations for source-level functional similarity detection," in *International Conference on Software Engineering*. IEEE, 2023.

[32] J. Svajlenko, J. F. Islam, I. Keivanloo, C. K. Roy, and M. M. Mia, "Towards a big data curated benchmark of inter-project code clones," in *International Conference on Software Maintenance and Evolution*. IEEE, 2014.

[33] T. Vislavski, G. Rakić, N. Cardozo, and Z. Budimac, "LICCA: A tool for cross-language clone detection," in *International Conference on Software Analysis, Evolution and Reengineering*. IEEE, 2018, pp. 512–516.

[34] E. Kodhai and S. Kanmani, "Method-level code clone detection through lwh (light weight hybrid) approach," *Journal of Software Engineering Research and Development*, vol. 2, no. 1, p. 12, 2014.

[35] J. R. Pate, R. Tairas, and N. A. Kraft, "Clone evolution: A systematic review," *Journal of software: Evolution and Process*, vol. 25, no. 3, 2013.

[36] M. Mondal, B. Roy, C. K. Roy, and K. A. Schneider, "An empirical study on bug propagation through code cloning," *Journal of Systems and Software*, vol. 158, 2019.

[37] R. Koschke, "Survey of research on software clones," in *Duplication, Redundancy, and Similarity in Software. Dagstuhl Seminar Proceedings*, vol. 6301. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2007.

[38] P. Estefó, R. Robbes, and J. Fabry, "Code duplication in ROS launch-files," in *International Conference of the Chilean Computer Science Society*. IEEE, 2015, pp. 1–6.

[39] S. McIntosh, M. Poehlmann, E. Juergens, A. Mockus, B. Adams, A. E. Hassan, B. Haupt, and C. Wagner, "Collecting and leveraging a benchmark of build system clones to aid in quality assessments," in *International Conference on Software Engineering*. ACM, 2014.

[40] G. Cardoen, T. Mens, and A. Decan, "A dataset of GitHub Actions workflow histories," in *International Conference on Mining Software Repositories*. ACM, 2024.

[41] T. Tsuru, T. Nakagawa, S. Matsumoto, Y. Higo, and S. Kusumoto, "Type-2 code clone detection for Dockerfiles," in *International Workshop on Software Clones*. IEEE, 2021.

[42] R. C. Merkle, "A digital signature based on a conventional encryption function," in *Conference on the theory and application of cryptographic techniques*. Springer, 1987, pp. 369–378.

[43] S. Bellon, R. Koschke, G. Antoniol, J. Krinke, and E. Merlo, "Comparison and evaluation of clone detection tools," *IEEE Transactions on Software Engineering*, vol. 33, no. 9, 2007.

[44] H. B. Mann and D. R. Whitney, "On a test of whether one of two random variables is stochastically larger than the other," *Ann. Math. Statist.*, vol. 18, no. 1, pp. 50–60, 03 1947.

[45] N. Cliff, "Dominance statistics: Ordinal analyses to answer ordinal questions," *Psychological bulletin*, vol. 114, no. 3, 1993.

[46] M. R. Hess and J. D. Kromrey, "Robust confidence intervals for effect sizes: A comparative study of Cohen's d and Cliff's delta under non-normality and heterogeneous variances," in *Annual Meeting of the American Educational Research Association*, 2004.

[47] M. Kim, V. Sazawal, D. Notkin, and G. Murphy, "An empirical study of code clone genealogies," in *European Software Engineering Conference*, 2005, pp. 187–196.

[48] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Springer, 2012.

[49] E. Kalliamvakou, G. Gousios, K. Blincoe, L. Singer, D. M. German, and D. Damian, "An in-depth study of the promises and perils of mining GitHub," *Empirical Software Engineering*, vol. 21, no. 5, 2016.